

Git/GitHub: Tutorial

UFR Sciences et Technologies

Lamri NEHAOUA

2025–2026

Configurer le Git

Git utilise votre nom et votre adresse e-mail pour étiqueter vos commits.
Configurez-les avec :

```
git config --global user.name "votre_nom"  
git config --global user.email "you_mail@example.com"
```

Affichage votre configuration :

```
git config --list
```

Pour modifier une valeur:

```
git config --global user.name "nouveau_nom"
```

Pour supprimer un paramètre:

```
git config --global --unset user.name
```

Créer et initialiser le Git

Créer un nouveau dossier pour votre projet:

```
mkdir myproject  
cd myproject
```

Initialiser le git (un dossier caché nommé `.git` est créé dans le projet):

```
git init
```

Git stocke toutes les informations nécessaires au suivi de vos fichiers et de leur historique dans ce dossier.

```
ls -a
```

Créer fichier

Le dépôt Git est vide. Ajouter un fichier à l'aide de votre éditeur de texte préféré et enregistrer-le sous le nom *index.html* dans le dossier de votre projet.

```
<!DOCTYPE html>
<html>
<head>
<title>Hello World!</title>
</head>
<body>

<h1>Hello world!</h1>
<p>This is the first file in my new Git Repo.</p>

</body>
</html>
```

Vérifiez si Git prend bien en compte le nouveau fichier :

```
git status
```

Git détecte le fichier *index.html*, mais il n'est pas suivi (**untracked**). Un fichier **untracked** de projet que Git ne surveille pas encore. Un fichier suivi (**tracked**) est un fichier dont Git surveille les modifications.

Préparer fichier

Pour qu'un fichier soit suivi, vous devez l'ajouter à la zone de préparation (**staging environment**). Cette zone sert de zone d'attente pour vos modifications. Elle permet d'indiquer précisément à Git les fichiers à inclure dans le prochain commit. Vous contrôlez ainsi le contenu de l'historique de votre projet. Pour ajouter un fichier à la zone de préparation, utilisez `git add`. Vérifier à nouveau par `git status`:

```
git add index.html
git status
```

Vous pouvez préparer toutes les modifications (nouveaux fichiers, fichiers modifiés et fichiers supprimés) en une seule fois.

```
git add --all
```

Si vous avez ajouté un fichier à la zone de préparation par erreur, vous pouvez le retirer avec :

```
git restore --staged index.html
```

Commit fichier

Un commit est un point de sauvegarde dans votre projet qui enregistre un instantané des fichiers à un moment précis. Un commit est toujours accompagné d'un message clair pour comprendre les changements. Pour enregistrer vos modifications préparées, utilisez:

```
git commit -m "your_message"
```

Vous pouvez ignorer l'étape de préparation (*staging*) avec l'option `-a`. Cette commande enregistre tous les fichiers modifiés et supprimés, mais pas les fichiers nouveaux ou non suivis.

```
git commit -a -m "message"
```

Pour écrire des messages de commit multilignes, n'utilisez pas l'option `-m`. L'éditeur par défaut s'ouvrira et vous pourrez rédiger un message sur plusieurs lignes :

```
git commit
```

Pour consulter l'historique des commits d'un dépôt utilisez `log`. Pour voir quels fichiers ont été modifiés dans chaque commit, utilisez l'option `--stat`

```
git log  
git log --stat
```

Etiquette Git

Dans Git, une étiquette (**tag**) est un marque-page pour un commit afin marquer les points importants de l'historique de votre projet, et de les partager avec votre équipe ou vos utilisateurs. Quelques types d'étiquettes courants :

- ▶ Version (**Release**): permettent d'indiquer quand le projet est prêt pour une publication, afin de retrouver cette version exacte par la suite.
- ▶ Jalons (**Milestone**): permettent de mettre en évidence les jalons importants, comme la finalisation d'une fonctionnalité majeure ou la correction d'un bug.
- ▶ Déploiement (**Deployment**): permettent de savoir quelle version de code à déployer.
- ▶ Correctifs (**Hotfixes**): permettent l'extraction d'une ancienne version et l'application du correctif.

Utilisez des étiquettes pour marquer les versions, les étapes importantes ou les points stables de votre projet. Créez les étiquettes après avoir réussi tous les tests ou avant de déployer/publier le code.

Etiquette Git

Créer une étiquette annotée (contient le nom, la date et un message):

```
git tag -a v1.0 -m "Version 1.0 release"
```

Pour afficher toutes les étiquettes d'un dépôt et leur détails :

```
git tag  
git show v1.0
```

Par défaut, les étiquettes existent uniquement en local. Pour que d'autres utilisateurs puissent voir vos étiquettes, vous devez les envoyer (push) vers votre dépôt distant (on le verra plus tard).

Pour supprimer une étiquette :

```
git tag -d v1.0
```


Comparer les changement

Afficher les modifications effectuées mais pas encore été préparées pour la validation.

```
git diff
```

affiche les modifications indexées (préparée) et prêtes à être validées.

```
git diff --staged
```

Afficher uniquement les commits effectués par un auteur spécifique, effectués au cours des deux dernières semaines :

```
git log --author="Nom_auteur "  
git log --since="2 weeks ago"
```

Branche Git

Dans Git, une branche est un espace de travail distinct où vous pouvez apporter des modifications et tester de nouvelles idées sans impacter le projet principal. Créer une nouvelle branche nommée *hello-world* et lister toutes les branches (dans le résultat, astérisque indique la branche courante):

```
git branch hello-world
git branch
```

Pour basculer vers une branche (vous pouvez désormais travailler sur la nouvelle branche sans affecter la branche principale *master*)

```
checkout hello-world
```

Editer votre fichier *index.html* et ajouter une image et sauvegarder.

```
<div></div>
```

Vérifiez l'état de la branche actuelle :

```
git status
```

Vous allez remarquer que des modifications ont été apportées à au fichier *index.html*, mais il n'est pas encore indexé (ou préparé) pour le commit. Ajouter les deux fichiers à la zone préparation pour cette branche et vérifiez l'état de la branche actuelle :

```
git add --all
```

Branche Git

Changer la branche vers `master` et lister les fichiers:

```
git checkout master  
ls
```

L'image ne fait pas partie de la branche `master`. Ouvrez le fichier HTML, le code est revenu à son état antérieur à la modification.

Pour supprimer une branche:

```
git branch -d hello-world
```

Fusionner les branches

Merge ou fusionner des branches dans Git signifie combiner les modifications d'une branche avec celles d'une autre.

Supposons qu'on doit corriger une erreur sur la branche `master` mais sans modifier directement la branche `master`, ni `hello-world`. Créer une nouvelle branche `emergency-fix` pour gérer la correction, modifier quelque chose dans le fichier `index.html` et sauvegarder. Vérifier l'état, préparer le fichier et commit.

```
git checkout -b emergency-fix
git status
git add index.html
git commit -m "updated index.html with emergency fix"
```

Placer vous sur la branche cible `master` et fusionner avec la branche `emergency-fix`.

```
git checkout master
git merge emergency-fix
```

Puisque la branche `emergency-fix` provient directement de `master`, et qu'aucune autre modification n'a été apportée à `master` pendant notre travail, Git la considère comme une continuation de `master`. Il peut donc effectuer une mise à jour rapide (*Fast-forward* ou aucun nouveau commit).

Annuler la fusion:

```
git merge --abort
```

Remote repository ou dépôt distant: est une version de votre projet hébergée sur Internet. GitHub est une plateforme populaire pour héberger des dépôts distants.

Rendez-vous sur GitHub et créez un compte gratuit. Une fois connecté, cliquez sur le bouton **Nouveau** pour créer un nouveau dépôt. Renseignez les informations du dépôt (nom, description, public/privé, etc.) et cliquez sur **Créer un dépôt**.

Lier votre dépôt Git local au dépôt distant sur GitHub. Pour communiquer entre les dépôts (local/distant), on peut utiliser le protocole HTTPS ou SSH. Dans ce tutoriel, on utilise HTTPS:

```
git remote add origin https://github.com/tonuser/votre_depot.git
```

GitHub vous permet de modifier des fichiers directement dans votre navigateur. Cliquez sur le fichier *README.md*, puis bouton **Modifier**. Modifier le fichier dans l'éditeur. Valider les modifications en ajoutez une courte description dans le champ **Message de validation**. Cliquer sur **Aperçu des modifications** pour voir les changements qui seront apportés au fichier. Sélectionnez **Créer une nouvelle branche** pour cette validation.

GitHub: Fetch, Pull and Merge

Lorsqu'on travaille en équipe sur un projet, il est important que chacun soit informé des dernières modifications. À chaque fois que vous commencez à travailler sur un projet, vous devez récupérer les modifications les plus récentes sur votre copie locale.

Télécharger les nouvelles données d'un dépôt distant sans modifier vos fichiers de travail ni vos branches. Elle vous permet de voir les modifications apportées par d'autres utilisateurs avant de fusionner ou de récupérer vos données.

```
git fetch origin
git status
```

Remarquer que vous un commit de retard sur la branche `origin/master` qui correspond à la mise à jour du fichier *README.md*. Vérifier encore en consultant le journal et afficher aussi les différences entre la branche `master` local (`()master`) et le `master` d'origine (`origin/master`):

```
git log origin/master
git diff origin/master
```

Si les mises à jour sont conformes aux attentes, fusionner les deux branches:

```
git merge origin/master
git status
```

GitHub: Fetch, Pull and Merge

Et si on mis à jour le dépôt local, sans passer par toutes ces étapes simplement par la commande `pull`? D'après la doc, `pull` combine les commandes `fetch` et `merge`. Apportez une autre modification au fichier *Readme.md* sur GitHub et mettez à jour le dépôt Git local:

```
git pull origin
```


GitHub: Fetch, Pull and Merge

Le transfert de nos modifications locales vers le dépôt distant, validé les modifications avec un `commit` puis un `push`.

```
git commit  
git push origin
```

Vous pouvez envoyer tous les tags locaux sur GitHub ou un tag spécifique:

```
git push --tags  
git push origin v1.0
```